

Meetkumar Saspara
Computer Science, University of North Texas
Arlington, TX
MeetkumarSaspara@my.unt.edu

April 5, 2026

Dr. Cynthia Pope
Department of Technical Communication
University of North Texas
Denton, TX 76203

Dear Dr. Pope,

Please find enclosed the formal report titled *Integrating Docker and Containerization into the UNT Computer Science Curriculum: A Feasibility Study and Recommendation*. This report was prepared for TECM 4180 and represents the culmination of a semester-long research effort involving a professional interview, a Qualtrics eSurvey of 28 UNT CS students, and five secondary sources spanning industry data, peer-reviewed research, and market analysis.

The report examines a documented gap between the containerization skills employers expect of software engineering graduates and what UNT's CS curriculum currently provides. Drawing on all three research methods, the report finds that Docker has become a baseline professional requirement, that students feel underprepared for DevOps expectations in the job market, and that integrating Docker into existing upper-division courses is both feasible and strongly supported by students. Based on these findings, the report recommends a phased two-semester implementation plan targeting Software Engineering (CSCE 3530), Operating Systems (CSCE 3600), and senior Capstone projects.

I appreciate your guidance throughout this project and welcome any feedback on this draft. Please do not hesitate to reach out if you have questions or require additional information.

Sincerely,

Meetkumar Saspara

Integrating Docker and Containerization into the UNT Computer Science Curriculum: A Feasibility Study and Recommendation

Prepared by
Meetkumar Saspara

Prepared for
Dr. Cynthia Pope
TECM 4180 — Technical Communication

University of North Texas
Department of Computer Science and Engineering
April 5, 2026

Abstract

This formal report investigates whether Docker and containerization tools should be integrated into the University of North Texas (UNT) Computer Science curriculum. The study employs a mixed-methods approach drawing on three research instruments: an interview with Jay Findoliya, a Senior Software Engineer at TechVista Solutions with over five years of containerization experience; a Qualtrics eSurvey of 28 UNT CS students examining their awareness of and preparedness for industry DevOps expectations; and a secondary literature review of five sources spanning industry adoption data, peer-reviewed educational research, and job market analysis.

Results indicate that containerization has become a non-negotiable baseline expectation in the software engineering industry, with 92% adoption among IT professionals. Despite this, UNT's CS curriculum does not formally cover Docker in any required course. Survey data confirms this gap: 18 of 28 respondents reported that Docker was never covered in their coursework, 64% feel underprepared for DevOps expectations in the job market, and 82% support adding formal Docker instruction. Interview findings corroborate these patterns, revealing that new graduates without Docker experience typically require two to three months of on-the-job training before contributing independently.

Based on these converging findings, the report recommends a phased two-semester implementation plan that embeds Docker into three existing upper-division courses — Software Engineering (CSCE 3530), Operating Systems (CSCE 3600), and senior Capstone projects — without adding credit hours or student costs. The integration follows the same model used when Git was adopted across CS courses and is supported by peer-reviewed research demonstrating that early DevOps exposure within existing coursework produces positive student outcomes.

Keywords: *Docker, containerization, computer science curriculum, DevOps, skills gap, higher education, UNT*

Table of Contents

Abstract	3
Introduction	5
Purpose.....	5
Scope.....	5
Methodology	5
Interview	6
Qualtrics eSurvey.....	6
Secondary Sources	6
Results	6
Interview Findings	6
eSurvey Findings	7
Secondary Research Findings.....	10
Discussion	11
Convergence of Findings	11
Feasibility.....	11
Limitations	11
Proposed Implementation Plan	11
Conclusion and Recommendation	12
References	13
Appendix	14
Appendix A: Interview Questions	14
Appendix B: eSurvey Questions.....	14

Introduction

Docker and containerization technologies have transformed modern software development by providing lightweight, portable, and reproducible environments that standardize how applications are built, tested, and deployed. Over the past decade, containerization has evolved from a specialized DevOps practice into a fundamental expectation for software engineers at all career levels. Industry data shows that container usage among IT professionals reached 92% in 2025, up from 80% the prior year (Docker, 2025), and the global application container market is projected to grow from \$5.85 billion to \$31.5 billion by 2030 (Grand View Research, 2024). Docker appears in over 60% of Kubernetes-related job postings, making it the single most frequently cited prerequisite technology across nearly 9,000 surveyed positions (Kube Careers, 2024).

Despite this industry-wide shift, the University of North Texas (UNT) Computer Science curriculum does not formally cover Docker or containerization in any required course. Students can earn their entire degree without ever writing a Dockerfile or managing a container. This gap has tangible consequences: students are entering internships and full-time roles without foundational skills that employers consider baseline, and companies are absorbing avoidable onboarding costs as a result.

This report investigates the extent of this curriculum gap and evaluates the feasibility of integrating Docker into existing upper-division courses at UNT. The study employs a mixed-methods approach, combining qualitative data from a professional interview, quantitative and qualitative data from a student eSurvey, and evidence from five secondary sources. The central argument is that Docker integration does not require new courses, additional credit hours, or significant financial investment — it requires intentional embedding of a widely-used tool into coursework that already touches the concepts containerization addresses.

Purpose

The purpose of this report is to present a comprehensive, research-supported case for integrating Docker and containerization tools into three existing UNT CS courses: Software Engineering (CSCE 3530), Operating Systems (CSCE 3600), and senior Capstone projects. The report synthesizes findings from primary and secondary research to demonstrate that this integration is warranted by industry demand, supported by students, and feasible within the current program structure.

Scope

This report covers the current state of containerization adoption in industry, the extent to which UNT's curriculum addresses this technology, student perceptions of their preparedness, professional perspectives on the skills gap, and a recommended implementation plan. The scope is limited to upper-division CS courses at UNT and does not address graduate-level programs or courses outside the Computer Science and Engineering department.

Methodology

This study used a mixed-methods research design combining primary research (an interview and an eSurvey) with secondary source analysis. Each method was chosen to provide a distinct angle on the research question, and the convergence of findings across all three methods strengthens the overall reliability of the conclusions.

Interview

A 30-minute semi-structured interview was conducted with Jay Findoliya, a Senior Software Engineer at TechVista Solutions, on February 8, 2026, via Zoom. Jay has over five years of experience working with containerized applications in production environments and regularly mentors junior developers on DevOps practices. The interview focused on industry expectations for containerization skills, onboarding challenges faced by new graduates, and practical recommendations for integrating Docker into university coursework. Ten open-ended questions were prepared in advance, covering topics ranging from daily Docker workflows to specific suggestions for course-level integration. The interview was authenticated via LinkedIn profile verification and email correspondence.

Qualtrics eSurvey

A Qualtrics eSurvey was distributed to UNT Computer Science students during the spring 2026 semester. The survey contained eight questions: one logic-based skip question, five closed-ended questions, one dependent follow-up question, and one open-ended question. Distribution channels included Canvas email to classmates in TECM 4180 and other CS courses, group chats with peers, the shared class Google Document, LinkedIn outreach, and QR codes posted on bulletin boards in the CS and engineering buildings. A total of 28 responses were collected over a one-week period. While 28 respondents represent a meaningful snapshot, this is a small fraction of the total UNT CS population; therefore, conclusions are directional rather than statistically generalizable. Respondents skewed toward upperclassmen, which aligns with the intended audience but limits representation of earlier students. Participation was voluntary, anonymous, and no personally identifiable information was collected.

Secondary Sources

Five secondary sources were selected through UNT's library database, Google Scholar, and direct industry publications to provide a range of perspectives. These include Docker's (2025) annual developer survey documenting industry adoption rates, a peer-reviewed study by Sánchez-Cifo et al. (2024) on introducing DevOps tools in CS education, a Kube Careers (2024) analysis of nearly 9,000 job postings, a Grand View Research (2024) market forecast, and Sorrel's (2019) practitioner account of implementing Docker in classroom settings. Two sources (Docker and Grand View Research) are produced by parties with commercial interest in containerization, which is noted as a limitation; however, the academic and job market sources corroborate the same trends independently.

Results

This section presents findings from each of the three research methods. Results are organized by data source and then synthesized in the Discussion section.

Interview Findings

Industry Expectations Far Exceed Current CS Curriculum. Jay Findoliya described containerization as a non-negotiable baseline at his company, noting that every project uses Docker from the first day and that it is no longer considered optional or advanced. When his company hires recent CS graduates, they expect basic Docker literacy, including understanding images versus containers, writing Dockerfiles, and using Docker Compose for multi-service applications. However, Jay noted that most new hires have never used containerization tools during their academic programs, creating a steep learning curve during the first months on the job.

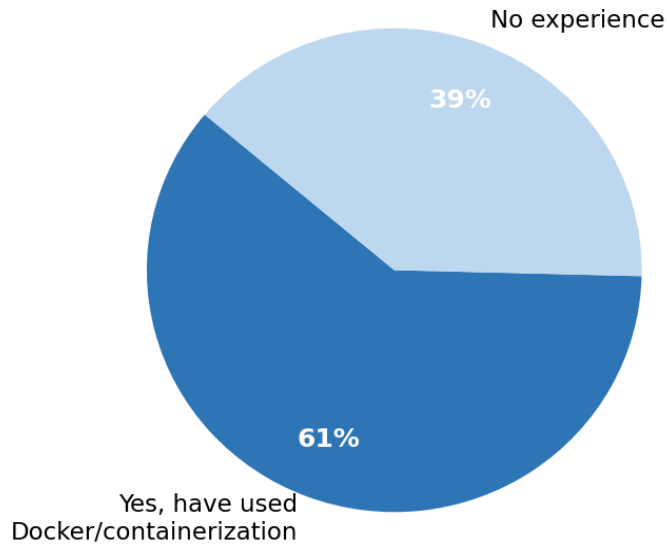
Onboarding Challenges Cost Time and Productivity. Jay provided specific examples of how this gap affects new developers. He described a scenario in which a new graduate spent three days setting up a local development environment because the graduate did not understand dependencies and version conflicts — a problem Docker resolves in minutes. According to Jay, developers without Docker experience typically need two to three months of hands-on work before they can independently containerize applications and troubleshoot common issues. This extended learning period delays their productivity and requires senior developers to provide what amounts to remedial technical training.

Practical Integration Opportunities Already Exist. Jay recommended integrating Docker into Software Engineering courses, explaining that students should use Docker for group projects to eliminate the common scenario where code works on one machine but not another. He also identified Capstone Projects and Operating Systems courses as natural fits because they already involve process isolation, resource management, and deployment pipelines. Rather than advocating for a standalone course, Jay emphasized that Docker should be woven into existing classes as a standard development tool, similar to how Git is taught across multiple courses.

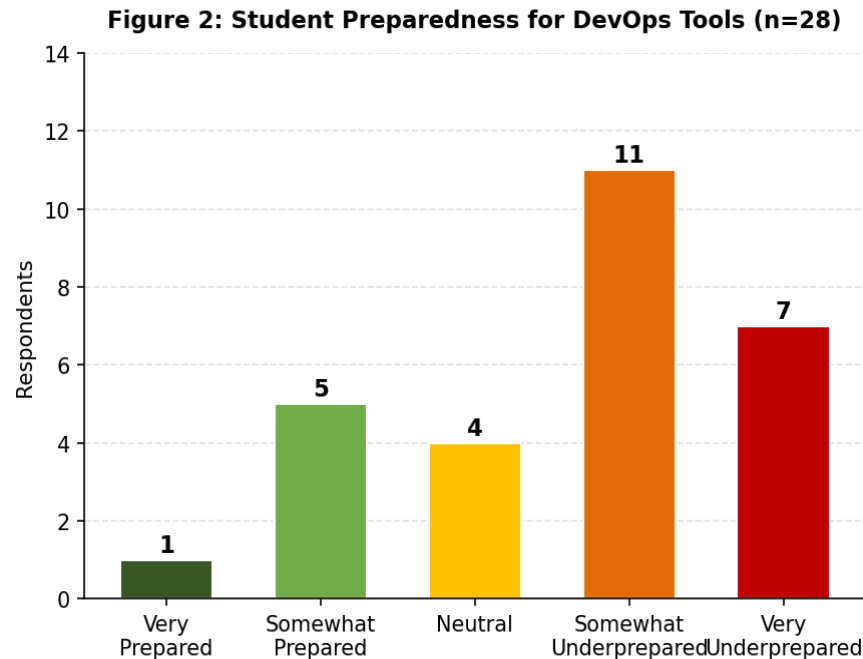
eSurvey Findings

Finding 1: Most Students Have Not Learned Docker Through UNT Coursework. When asked whether they had ever used Docker or a containerization tool, 17 of 28 respondents (61%) answered yes. However, the follow-up question revealed a critical detail: of those 17, none identified a UNT course as their primary learning source. Respondents credited personal or side projects (8), online self-study (6), and internships or jobs (3). The remaining 11 respondents (39%) reported no experience with containerization tools at all. When asked specifically whether any UNT CS course had covered Docker, 18 respondents stated it had never been covered, and 7 said it was only briefly mentioned without depth. Only 3 indicated encountering it in even one course. See Figure 1.

Figure 1: Prior Docker / Containerization Experience (n=28)



Finding 2: Students Feel Underprepared for DevOps Expectations. As shown in Figure 2, 18 of 28 respondents (64%) indicated they feel either somewhat underprepared or very underprepared for the DevOps expectations they encounter in internship and job applications. Only 6 respondents felt at least somewhat prepared, and a single respondent described themselves as very prepared. Supporting this finding, 20 of 28 respondents (71%) reported having encountered a job posting or internship listing that required Docker or containerization knowledge, with 9 saying they see such requirements frequently (see Figure 4). This confirms that the skills gap is not hypothetical — students are directly experiencing it as they apply for opportunities.



Finding 3: Strong Student Support for Formal Docker Instruction. As illustrated in Figure 3, 23 of 28 respondents (82%) agreed or strongly agreed that Docker and containerization should be formally taught as part of the UNT CS curriculum. Only 2 respondents expressed any disagreement. The open-ended question yielded further reinforcement: Docker was the most commonly mentioned tool by name, appearing in 9 of the 28 responses. Other frequently cited tools included Git/GitHub workflows (5), CI/CD pipelines (4), and Linux command-line fundamentals (3). Several respondents described Docker as something they were expected to already know when applying for positions.

Figure 3: Should Docker Be Formally Taught at UNT? (n=28)

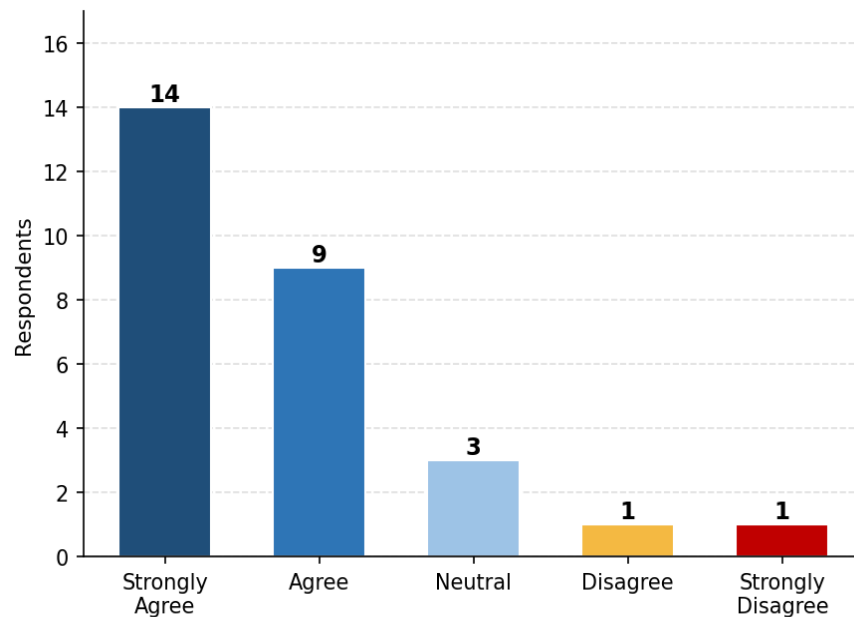
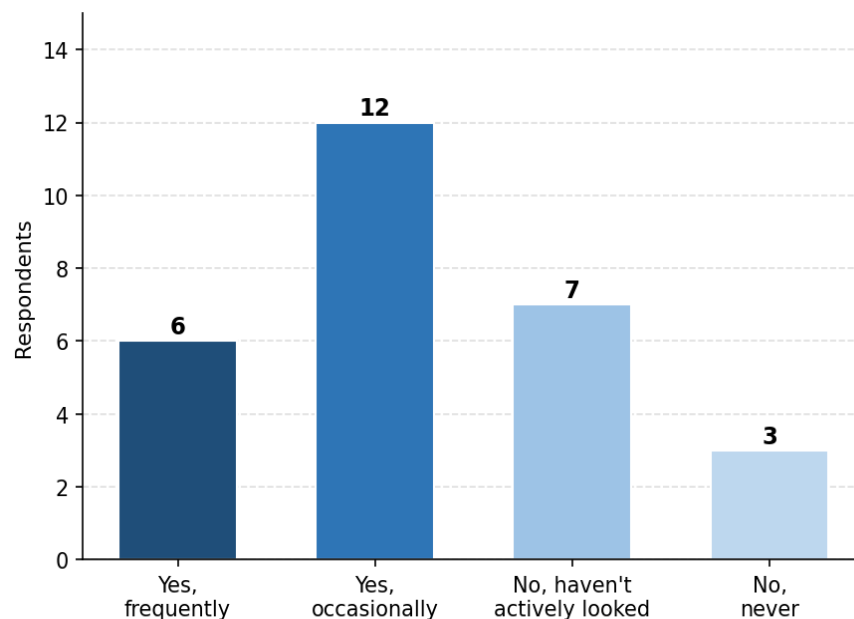


Figure 4: Docker in Job/Internship Listings (n=28)



Secondary Research Findings

Containerization Is Now a Baseline Industry Expectation. Docker's (2025) State of Application Development report documents that container usage among IT professionals reached 92%, representing the largest single-year adoption increase of any surveyed technology. Grand View Research (2024) contextualizes this within broader market forces, projecting the container market to grow at a 33.5% compound annual growth rate through 2030. The Kube Careers (2024) job posting analysis reinforces the employer perspective: Docker appears in over 60% of

Kubernetes-related postings across 8,772 positions, and professionals with containerization skills earn an average of \$158,134 globally, with North American positions averaging \$169,627.

A Measurable Skills Gap Exists. Sánchez-Cifo et al. (2024), in a peer-reviewed study at the ACM ICSE-SEET conference, found that CS students consistently struggle to connect academic concepts to the practical DevOps skills industry requires. The demand for engineers with containerization experience continues to rise faster than programs produce qualified candidates. Their study found that structured early exposure to DevOps tools produced student outcomes described as extremely positive.

Docker Integration Is Pedagogically Feasible. Sorrel (2019) documents a practitioner's experience integrating Docker into existing database and web server coursework, emphasizing that Docker's cross-platform nature eliminates the environment inconsistency problems that already consume class time. Rather than requiring a standalone course, Sorrel's approach treats Docker as a development tool woven into existing instruction. Sánchez-Cifo et al. (2024) similarly found that introducing DevOps tools within a Software Engineering fundamentals course was both logistically manageable and educationally effective.

Discussion

The findings from all three research methods converge on a consistent set of conclusions. This section interprets those findings, addresses limitations, and connects the results to the proposed implementation.

Convergence of Findings

The interview, eSurvey, and secondary sources each independently confirm that a gap exists between what UNT teaches and what the software engineering industry expects regarding containerization. Jay Findoliya's professional experience — watching new graduates struggle with tasks that Docker handles in minutes — is directly reflected in the eSurvey data, where 64% of students report feeling underprepared for DevOps expectations. The secondary literature provides the macro-level context: with 92% industry adoption and Docker appearing in the majority of job postings, this is not a niche skill but a professional baseline. The consistency across qualitative, quantitative, and published evidence lends strong credibility to the conclusion that curricular change is warranted.

Feasibility

A key concern with any curricular change is whether it creates additional burden for students, faculty, or departmental budgets. The evidence suggests this concern is minimal for Docker integration specifically. Docker Desktop is free for educational use and runs on any modern laptop, eliminating hardware costs. The proposed approach modifies existing assignments rather than creating new courses, which means no curriculum committee approval for new course numbers is required. Both Sorrel (2019) and Sánchez-Cifo et al. (2024) document successful integration of Docker into existing courses without disruption, and Jay Findoliya's

recommendation to model the rollout after Git adoption across CS courses provides a practical, tested framework.

The primary investment is a two-day faculty workshop before the pilot semester. This is a modest ask relative to the documented benefits: students graduating with hands-on experience in a tool that appears in the majority of software engineering job postings, reduced onboarding costs for employers, and strengthened program reputation in the DFW tech market.

Limitations

Several limitations should be noted. The eSurvey sample of 28 respondents, while meeting the assignment threshold, is a small fraction of the total UNT CS student population. Findings are therefore directional rather than statistically generalizable. Respondents skewed toward upperclassmen, limiting representation of earlier students. The interview represents a single professional perspective; additional interviews with hiring managers, faculty, and UNT IT staff would strengthen the recommendation. Two secondary sources (Docker and Grand View Research) come from organizations with commercial interest in containerization, though their findings are corroborated by independent academic and job market data.

Proposed Implementation Plan

Based on the convergence of all findings, the following phased two-semester plan is recommended:

Phase	Task	Target Date
1	Present proposal to CSE leadership	May–Jun 2026
2	Faculty Docker workshop (two days)	Jul–Aug 2026
3	Pilot in one section of CSCE 3530	Fall 2026
4	Collect feedback; refine materials	Dec 2026–Jan 2027
5	Expand to CSCE 3600 and Capstone	Spring 2027
6	Evaluate outcomes; recommend adoption	May 2027

Table 1: Proposed Two-Semester Implementation Timeline

Conclusion and Recommendation

This report set out to determine whether Docker and containerization tools should be integrated into the UNT Computer Science curriculum, and if so, how. The evidence from every source consulted — a professional interview, a student eSurvey, and five secondary sources — points in the same direction.

Docker is an industry standard. Container usage has reached 92% among IT professionals, Docker appears in over 60% of relevant job postings, and the container market is projected to grow at a 33.5% annual rate through 2030. Students confirm the gap: 18 of 28 survey respondents said Docker was never covered in their UNT courses, 64% feel underprepared for

DevOps expectations, and 82% support formal instruction. An experienced software engineer corroborates these findings, describing the tangible productivity costs of onboarding graduates who lack containerization skills.

The proposed solution is practical and low-cost. Docker Desktop is free for educational use, the integration modifies existing assignments rather than creating new courses, and peer-reviewed research supports the pedagogical approach. The recommended two-semester implementation plan — beginning with a faculty workshop in Summer 2026, piloting in one section of CSCE 3530 in Fall 2026, and expanding to CSCE 3600 and Capstone by Spring 2027 — follows the same model used when Git was adopted across CS courses.

Recommendation: The Department of Computer Science and Engineering should approve and implement the phased Docker integration plan outlined in this report. This update would improve student employability, strengthen the program’s reputation as a source of job-ready talent in the DFW tech market, and reduce the gap between what students learn in the classroom and what employers expect on day one. The data from three independent research methods supports this conclusion, and the implementation path carries minimal risk, cost, and disruption to existing program structures.

References

- Docker. (2025, July 11). 2025 Docker state of app dev: Key insights revealed. *Docker Blog*.
<https://www.docker.com/blog/2025-docker-state-of-app-dev/>
- Findoliya, J. (2026, February 8). Personal interview [Zoom].
- Grand View Research. (2024). Application container market size | Industry report, 2030.
<https://www.grandviewresearch.com/industry-analysis/application-container-market>
- Kube Careers. (2024). *The state of Kubernetes jobs in 2024 Q3*. <https://kube.careers/state-of-kubernetes-jobs-2024-q3>
- Sánchez-Cifo, M. Á., Bermejo, P., & Navarro, E. (2024). Introducing computer science undergraduate students to DevOps technologies from software engineering fundamentals. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training* (pp. 1–2). Association for Computing Machinery.
<https://doi.org/10.1145/3639474.3640071>
- Saspara, M. (2026). Docker and containerization in the UNT CS curriculum [Qualtrics eSurvey]. Distributed via Canvas, LinkedIn, and group chats, February–March 2026.
- Sorrel, H. (2019, November 3). Docker in the classroom (part 1). *Medium*.
<https://sorrelharriet.medium.com/docker-in-the-classroom-part-1-3d86fea9160c>

Appendix

Appendix A: Interview Questions

The following ten questions were prepared for the interview with Jay Findoliya on February 8, 2026:

1. Can you describe your typical workflow when developing or deploying software applications, and where Docker or containerization fits into that process?
2. When your company hires entry-level software engineers or recent CS graduates, what containerization skills or knowledge do you expect them to have, if any?
3. What challenges have you observed when onboarding new developers who lack experience with Docker or containerization technologies?
4. In your experience, how long does it typically take for a developer with no containerization background to become proficient enough to work effectively with Docker in a team environment?
5. Beyond Docker itself, what related technologies or concepts (such as Kubernetes, CI/CD pipelines, or orchestration tools) do you consider essential for modern software engineers to understand?
6. If you were designing a CS course that introduced containerization, which specific topics or hands-on activities would you prioritize to give students the most practical, career-relevant experience?
7. How has containerization changed the way your team handles issues like environment consistency, dependency management, or collaboration across different operating systems?
8. What are the most common mistakes or misconceptions you've seen developers make when first learning to work with containerized environments?
9. From your perspective, which upper-division CS courses (such as Software Engineering, Operating Systems, or Capstone Projects) would benefit most from integrating Docker as a standard development tool, and why?
10. Looking ahead at industry trends, how critical do you believe containerization skills will be for software engineering graduates entering the job market over the next 3–5 years?

Appendix B: eSurvey Questions

The following eight questions were included in the Qualtrics eSurvey distributed to UNT CS students:

Q1 (Logic/Skip): Have you ever used Docker or any containerization tool (e.g., Podman, Kubernetes)? [Yes / No] — If No, skip to Q3.

Q2 (Dependent on Q1): Where did you primarily learn to use Docker or containerization tools?
[A UNT course / A personal or side project / An internship or job / Online tutorials or self-taught
/ A hackathon or student organization]

Q3: How familiar are you with the concept of containerization in software development? [Very familiar / Somewhat familiar / Slightly familiar / Not familiar at all]

Q4: Have any of your UNT CS courses covered Docker or containerization as part of the curriculum? [Yes, in multiple courses / Yes, in one course / Briefly mentioned but not taught in depth / No, never]

Q5: How prepared do you feel by the UNT CS curriculum for DevOps tools and practices expected in internships and jobs? [Very prepared / Somewhat prepared / Neutral / Somewhat underprepared / Very underprepared]

Q6: Do you believe Docker and containerization should be formally taught in the UNT CS curriculum? [Strongly agree / Agree / Neutral / Disagree / Strongly disagree]

Q7: Have you ever seen a job posting or internship listing that required Docker or containerization knowledge? [Yes, frequently / Yes, occasionally / No, but I haven't actively looked / No, never]

Q8 (Open-ended): What is one technology or industry tool you wish the UNT CS program had taught you before you started applying for internships or jobs?